

LABORATORY TECHNICAL REPORT: DESIGN AND IMPLEMENTATION OF A HYBRID HIGH-PERFORMANCE COMPUTING (HPC) AND BIG DATA CLUSTER

Course Title: Distributed Systems & Parallel Computing Algorithms Laboratory

Development Environment: Ubuntu Linux OS Architecture (1 Master Node, 2 Worker Nodes)

Document Variant: Academic Final Submission Template (Word Native Import Layout)

Section 1: Mini HPC Cluster Infrastructure Design & Configuration

The implementation begins with deploying an isolated virtual local area network spanning three virtual machines running Ubuntu Linux server distributions. Resource segregation assigns administrative control tasks to a designated system coordinator node (Master Node) while execution processes are offloaded to distributed workers (Worker 1 and Worker 2).

1.1 Network Hostname Resolution Mapping (/etc/hosts)

To eliminate runtime name-resolution overhead and prevent dynamic routing faults, hostfiles were updated across all cluster components to map static private addresses directly to internal aliases.

[IMAGE PLACEHOLDER: 01_network_hosts_setup.png / image_773545.jpg]

Figure 1.1: Verification of static IP assignments for master, worker1, and worker2 nodes within the host terminal system.

1.2 Inter-Node Interface Connectivity Evaluation

Network path verification was executed across the cluster grid infrastructure using uniform packet verification methods. Low network latency and zero percent loss metrics confirmed complete stability across the communication backbone.

[IMAGE PLACEHOLDER: 02_cluster_connectivity_test.png / image_7735f8.jpg]

Figure 1.2: Continuous verification streams from the master terminal interface testing communication loopback to worker nodes.

1.3 Secure Cryptographic Passwordless SSH Integration

Distributed job control layers require automated, credential-free operational access to remote

cluster workers. Secure RSA public-private key pairings were generated and transmitted across the security boundary to eliminate active text prompts.

[IMAGE PLACEHOLDER: 03_ssh_worker1.png / image_773184.jpg]

Figure 1.3: Remote login test sequence to worker1 without manual password interaction.

[IMAGE PLACEHOLDER: 04_ssh_worker2.png / image_7731be.jpg]

Figure 1.4: Remote login test sequence to worker2 demonstrating credential-free access.

1.4 OpenMPI Parallel Optimization & Core Allocation Map

Message Passing Interface execution capabilities were installed via openmpi-bin and libopenmpi-dev packages. Core allocation boundaries were restricted within a cluster layout parameters file named hostfile, defining core limitations across system processes:

master slots=2

worker1 slots=2

worker2 slots=2

[IMAGE PLACEHOLDER: 05_openmpi_installation.png / 06_hostfile_configuration.png / image_7735bf.jpg / image_77359a.jpg]

Figure 1.5: System configuration parameters and library version outputs for OpenMPI cluster runtimes.

1.5 Cluster Parallel Runtime Verification (MPI Hello World)

To verify the parallel execution environment across separate nodes, an automated testing script was deployed using mpi4py to confirm communication across all nodes.

[IMAGE PLACEHOLDER: 07_mpi_hello_world.png / 08_mpi_python_validation.png / image_7738fe.jpg / image_7738c6.jpg]

Figure 1.6: Parallel process verification outputs showing successful multi-node communication across master and worker instances.

Section 2: Parallel Machine Learning Optimization (Digits Dataset Matrix)

Computational cluster performance was benchmarked using distributed model training processes over the scikit-learn handwritten text matrix framework. Data splitting methods separated feature and label fields into segmented packets managed using comm.scatter, returning final analytical measurements to base process 0 via comm.gather.

[IMAGE PLACEHOLDER: 09_digits_dataset.png / 10_digits_script.png / 11_digits_execution.png / image_7738a4.jpg / image_773921.jpg]

Figure 2.1: Parallel Python ML Training Logic & Job Tracing Logs showing concurrent sample splitting.

Section 3: Parallel Bioinformatics Analysis (GC Content Sequence Evaluation)

The cluster's scientific compute layer was tested using a parallel data pipeline designed to parse

FASTA genomic format structures from sequences.fasta and calculate nucleotide values concurrently across available compute slots.

GC Percentage (%) = [Count(G) + Count(C)] / Total Base Pairs Count × 100

Target Execution Node	Process Allocation Rank	Target Genomic Sequence Processed	Calculated GC Content Metric (%)
master	Rank 1	ATATATATATATATATATAT AT	0.00%
worker1	Rank 2	GGGCCCCGGGCCCGG GCCC	100.00%
worker1	Rank 3	ATGCCGTAGCTAGCT AGCTA	50.00%
worker2	Rank 4	CGCGCGCGCGCGAT ATATA	63.16%
master	Rank 0	ATGCGCGATATCGCG CGCTA	60.00%
worker2	Rank 5	GCGCGCGCGCGCGC GCGCGC	100.00%

[IMAGE PLACEHOLDER: 12_bioinformatics_dataset.png / 13_bioinformatics_script.png / 14_bioinformatics_execution.png / image_7792da.jpg]

Figure 3.1: Runtime execution logs tracking distributed sequence parsing across the worker grid nodes.

Section 4: Hybrid Container Frameworks (Docker Swarm & Apache Spark Architecture)

The second stage of development updates the cluster from static parallel environments into a dynamic, containerized ecosystem. Docker Swarm was configured as the infrastructure orchestration manager to deploy scalable Apache Spark services.

4.1 Swarm Infrastructure Activation Logs

Container swarm operations were activated using the master terminal hub. Subordinate worker assets integrated with the cluster using validation access tokens, monitored via the docker node ls resource control utility.

[IMAGE PLACEHOLDER: 15_docker_swarm_init.png / 16_docker_swarm_join.png / 17_docker_swarm_nodes.png / image_773501.jpg / image_773201.jpg / image_7731e2.jpg]

Figure 4.1: Status outputs confirming operational swarm activation and healthy worker nodes connectivity.

4.2 Service Stack Deployment Troubleshooting & Image Conflict Resolution

Deploying initial infrastructure models using complex third-party system components resulted in standard startup exceptions (No such image errors) on independent nodes because of missing

localized images. Refactoring resource configurations to pull official, fully mirrored apache/spark:latest environments corrected sync failures and restored service stability (Replicas: 1/1).

[IMAGE PLACEHOLDER: 18_spark_stack_deployment.png / image_773d01.jpg / image_773d60.png / image_773d82.jpg / image_773ce4.jpg / image_773d3e.jpg]

Figure 4.2: Troubleshooting progression showing corrected service failures and successful access to the Spark dashboard Web UI.

4.3 PySpark Large-Scale Transcriptomic Expression Analysis

Using the active Spark cluster, predictive data processing was completed against an expression profiling file named leukemia_expression.csv. Integrated VectorAssembler components combined input metrics into unified matrices to train predictive LogisticRegression classification structures.

[IMAGE PLACEHOLDER: 19_pyspark_installation.png / 20_gene_expression_dataset.png / 21_gene_expression_script_created.png / 22_gene_expression_results.png / image_7740ff.jpg / image_7740a5.jpg / image_774061.jpg / image_77399b.jpg / image_77397e.png / image_773945.jpg]

Figure 4.3: Execution pipeline and analytics logs demonstrating 100% predictive modeling accuracy across patient records.

Section 5: Summary Evaluation & Technical Conclusion

This technical implementation successfully engineered a multi-tiered computing infrastructure. It bridges the gap between classic parallel systems frameworks (OpenMPI) used for atomic computational analysis and containerized data engineering services (Docker Swarm & Apache Spark) used for large-scale classification pipelines. This integration provides a scalable, stable, and highly adaptable computing grid designed to handle demanding data science and scientific computing workloads.